



Unit IV: Packages

PL/SQL Packages

PPT PRESENTED BY

NAME : Srinivas Rao Tammireddy

ROLL NO : 257Y1D5805

YEAR : I Year I Semester

ACADEMIC YEAR : 2025 - 2026

What is a PL/SQL Package?

A PL/SQL Package is a schema object that groups logically related PL/SQL types, variables, constants, subprograms, and cursors. It consists of two distinct components compiled separately.

Two-Part Architecture

- 1. Package Specification (Spec):** The public interface / header. Declares public variables, cursors, types, and subprogram signatures. Visible to other database programs.
 - 2. Package Body:** The private implementation. Contains the code for subprograms declared in the specification, as well as private variables, types, and helper methods. Hidden from outer scope.
- **Separate Compilation:** You can change and re-compile the body without compiling the specification, preventing invalidation of dependent objects.

Core Benefits of Packages

- **Modular Development:** Groups related business features into clean libraries (e.g., HR_PKG, FINANCE_PKG).
- **Encapsulation (Information Hiding):** Keeps helper functions and variables private inside the body, protecting the API implementation.
- **Session Persistent State:** Package variables persist for the duration of a user's database session, acting as in-memory global state.
- **High Performance:** The entire package is loaded into memory on the first call, reducing database page reads for subsequent subprogram runs.

Definer Rights vs. Invoker Rights

Packages run under different security context privileges, controlled by AUTHID:

Definer Rights (AUTHID DEFINER)

- **Default Privilege Behavior:** Run with the schema privileges of the user who CREATED (defined) the package.
- **Security Encapsulation:** Allows a user to perform modifications on tables they cannot access directly, by calling a procedure owned by a privileged creator.
- **Name Resolution:** Unqualified table names resolve to the definer's schema.
- **Execution context:** Ideal for secure, transactional routines where table data modifications are highly regulated.

Invoker Rights (AUTHID CURRENT_USER)

- **Dynamic Privileges:** Run with the schema privileges of the user currently executing (invoking) the package.
- **Name Resolution:** Unqualified table references resolve to the invoker's current schema. The same package query fetches different data depending on who runs it.
- **Code sharing:** Ideal for utility packages (e.g. string formatting, mathematical tools) shared across multiple schemas.
- **Security constraint:** Prevents users from escalating privileges using malicious package code.

Managing Packages: Invalidation & Catalog

How Oracle monitors package status, invalidation, and dependencies:

Database Catalog Queries

- **Finding Packages:** Metadata is stored in catalog views like `USER_OBJECTS` and `USER_SOURCE`:
`SELECT object_name, status FROM user_objects WHERE object_type = 'PACKAGE';`
- **Status Monitoring:** Status can be 'VALID' or 'INVALID'. If a dependent table changes, the package becomes invalid and must be re-compiled.
- **Describing Packages:** Use `DESCRIBE` command in SQL*Plus to check public signatures:
`DESCRIBE hr_pkg;`

Timestamp vs. Signature Validation

When a package is modified, Oracle determines if dependent programs must be recompiled using two methods:

1. **Timestamp Method (Default):** Compares compile timestamps. If the package spec changes, all calling programs are immediately marked invalid and must re-compile. Causes cascading invalidations.
2. **Signature Method:** Compares subprogram parameters and data types (signatures). Calling programs remain valid as long as parameter signatures are unchanged, avoiding rebuild loops.

Applications of Packages

PL/SQL packages are utilized to build modular, secure database interfaces:

Enterprise business API libraries

Groups related workflows into single libraries (e.g. Ledger_Pkg, Inventory_Pkg). Standardizes how client software executes transactions, enforcing business rules.

Session Cache Managers

Package variables persist throughout a database connection. They are utilized to cache static data (e.g., system settings, user permissions) in memory, avoiding redundant table queries.

System Integration Modules

Encapsulates complex database integrations, such as calling external APIs, parsing XML/JSON, or managing TCP connections inside private package body routines.

Utility Toolkits

Groups helper subprograms for data mapping, error logging, and date arithmetic. Oracle built-in packages like DBMS_SQL and UT_FILE are classic examples.



THANK YOU

Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management